

# Module 01: Foundations of Spring AI, Architecture & Model Providers

## Modern Generative AI in Enterprise Java (2026 Edition)

**Instructor:** AI Curriculum Engineering Team

**Tech Stack:** Java 25/27 (LTS), Spring Boot 3.4/4.x, Spring AI 1.0+, Ollama

# 1. Why Generative AI in Java?

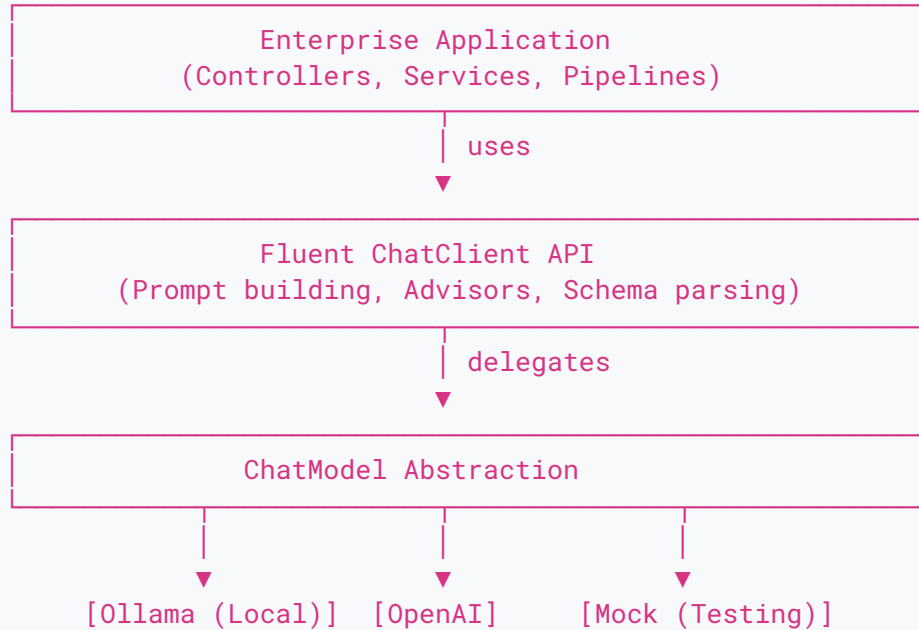
Enterprise software runs on Java. Historically, GenAI tooling was concentrated in Python notebooks. However, production enterprise applications require:

- **Type Safety:** Compile-time schema enforcement.
- **High Concurrency:** Thousands of concurrent user sessions without GIL bottlenecks.
- **Observability:** Direct integration with Micrometer, OpenTelemetry, and Jaeger.
- **Ecosystem Integration:** Security, transaction boundaries, and microservice meshes.

Spring AI bridges this gap by bringing idiomatic Spring patterns to AI engineering.

## 2. Spring AI Core Architecture

Spring AI decouples your business logic from underlying foundation model providers:



## 3. ChatModel vs. ChatClient

### The Evolution:

- **ChatModel (Low-Level):**
  - Direct 1-to-1 mapping with vendor APIs.
  - Takes raw **Prompt** objects containing lists of **Message** instances.
  - Returns **ChatResponse**.
- **ChatClient (High-Level / Modern):**
  - Fluent, declarative builder API introduced in Spring AI 1.x.
  - Configurable default system messages, default advisors (logging, memory, security).
  - Strongly-typed structured output conversions (**entity()**).

## 4. Fluent ChatClient in Action

```
@Service
public class AssistantService {

    private final ChatClient chatClient;

    public AssistantService(ChatClient.Builder chatClientBuilder) {
        this.chatClient = chatClientBuilder
            .defaultSystem("You are a senior Java architect.")
            .defaultAdvisors(new SimpleLoggerAdvisor())
            .build();
    }

    public String askQuestion(String question) {
        return chatClient.prompt()
            .user(question)
            .call()
            .content();
    }
}
```

## 5. Model Portability & Dynamic Routing

Never hardcode your application to a single LLM vendor. Use the **Model Routing Pattern**:

```
@Service
public class ModelRoutingService {
    private final Map<String, ChatModel> models;

    public ModelRoutingService(ApplicationContext context) {
        this.models = context.getBeansOfType(ChatModel.class);
    }

    public ChatClient getClient(String provider) {
        ChatModel selected = models.getOrDefault(provider, defaultModel);
        return ChatClient.builder(selected).build();
    }
}
```

Switch between local Ollama (`llama3.2`), OpenAI (`gpt-4o`), or offline Mocks with zero code modifications!

## 6. Reactive Token Streaming via SSE

For interactive interfaces, users expect streaming responses (sub-second TTFT - Time To First Token):

```
@GetMapping(value = "/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<ServerSentEvent<String>> stream(@RequestParam String prompt) {
    return chatClient.prompt()
        .user(prompt)
        .stream()
        .content()
        .map(token -> ServerSentEvent.<String>builder()
            .data(token)
            .build());
}
```

Spring WebFlux & Spring AI seamlessly stream chunks without thread starvation.

## 7. Concurrency with Java 25/27 Virtual Threads

Modern Java provides lightweight Virtual Threads ([Project Loom](#)). When querying multiple LLMs concurrently:

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {  
    List<CompletableFuture<ProviderResult>> futures = models.stream()  
        .map(model -> CompletableFuture.supplyAsync(  
            () -> queryModel(model, prompt), executor  
        ))  
        .toList();  
  
    CompletableFuture.allOf(futures.toArray(new CompletableFuture[0])).join();  
}
```

- Millions of virtual threads can be spawned with negligible memory overhead.
- Long I/O waits during LLM inference do not block OS platform threads.

## 8. Homework 01 Assignment & Hands-on Lab

### "Multi-Model Health & Comparison Service"

1. Navigate to `module-01-foundations-and-models/homework/starter`.
2. Inspect `MultiModelComparisonService.java`.
3. Complete the `compareAcrossModels` method using Virtual Threads.
4. Ensure provider timeouts and connection errors are isolated.
5. Verify passing tests with `mvn test`.